

Java Collections - 10 CRUD Case Studies (List, Set, Map, Queue)

Author: Thaneshwara M

===== 1) Employee Management - List (ArrayList) =====

// File: EmployeeListCRUD.java

package com.collectiondemo;

import java.util.*;

// ----- MODEL CLASS -----

```
class Emp {
    int id;
    String name;
    int age;
    double salary;
    String desig;
    public Emp(int id, String name, int age, double salary, String desig) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.salary = salary;
        this.desig = desig;
    }
    void display() {
        System.out.println("-----");
        System.out.println("ID      : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Age    : " + age);
        System.out.println("Salary : " + salary);
        System.out.println("Desig  : " + desig);
        System.out.println("-----");
    }
}
```

// ----- MAIN CLASS -----

```
public class EmployeeListCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Emp> empList = new ArrayList<>();
        int choice;
        do {
            System.out.println("\n==== EMPLOYEE MANAGEMENT (ArrayList) ====");
            System.out.println("1. Add Employee (CREATE)");
            System.out.println("2. View All Employees (READ)");
            System.out.println("3. Update Salary by ID (UPDATE)");
            System.out.println("4. Delete Employee by ID (DELETE)");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1: // CREATE
                    System.out.print("Enter ID : ");
                    int id = sc.nextInt();
                    System.out.print("Enter Name : ");
                    String name = sc.next();
                    System.out.print("Enter Age : ");
                    int age = sc.nextInt();
                    System.out.print("Enter Salary: ");
                    double sal = sc.nextDouble();
                    System.out.print("Enter Desig: ");
                    String desig = sc.next();
                    empList.add(new Emp(id, name, age, sal, desig));
                    System.out.println("Employee Added.");
                    break;
                case 2: // READ
                    if (empList.isEmpty()) {
                        System.out.println("No employees to display.");
                    } else {
```

```

        System.out.println("\n--- Employee Details ---");
        for (Emp e : empList) {
            e.display();
        }
    }
    break;
case 3:    // UPDATE
    System.out.print("Enter ID to update salary: ");
    int uid = sc.nextInt();
    boolean found = false;
    for (Emp e : empList) {
        if (e.id == uid) {
            System.out.print("Current Salary = " + e.salary +
                             ". Enter Increment Amount: ");
            double inc = sc.nextDouble();
            e.salary += inc;
            System.out.println("Salary updated. New Salary = " + e.salary);
            found = true;
            break;
        }
    }
    if (!found) {
        System.out.println("Employee not found.");
    }
    break;
case 4:    // DELETE
    System.out.print("Enter ID to delete: ");
    int did = sc.nextInt();
    Emp toRemove = null;
    for (Emp e : empList) {
        if (e.id == did) {
            toRemove = e;
            break;
        }
    }
    if (toRemove != null) {
        empList.remove(toRemove);
        System.out.println("Employee removed.");
    } else {
        System.out.println("Employee not found.");
    }
    break;
case 5:
    System.out.println("Thank you!");
    break;
default:
    System.out.println("Invalid choice.");
}
} while (choice != 5);
sc.close();
}
}

```

===== 2) Student Management - List (ArrayList) =====

// File: StudentListCRUD.java

package com.collectiondemo;

import java.util.*;

// ----- MODEL CLASS -----

```

class Student {
    int id;
    String name;
    String course;
    int marks;
    public Student(int id, String name, String course, int marks) {
        this.id = id;
    }
}

```

```

        this.name = name;
        this.course = course;
        this.marks = marks;
    }
    void display() {
        System.out.println("-----");
        System.out.println("ID      : " + id);
        System.out.println("Name    : " + name);
        System.out.println("Course  : " + course);
        System.out.println("Marks   : " + marks);
        System.out.println("-----");
    }
}

// ----- MAIN CLASS -----
public class StudentListCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Student> stuList = new ArrayList<>();
        int choice;
        do {
            System.out.println("\n==== STUDENT MANAGEMENT (ArrayList) ====");
            System.out.println("1. Add Student (CREATE)");
            System.out.println("2. View All Students (READ)");
            System.out.println("3. Update Marks by ID (UPDATE)");
            System.out.println("4. Delete Student by ID (DELETE)");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1: // CREATE
                    System.out.print("Enter ID      : ");
                    int id = sc.nextInt();
                    System.out.print("Enter Name    : ");
                    String name = sc.next();
                    System.out.print("Enter Course  : ");
                    String course = sc.next();
                    System.out.print("Enter Marks   : ");
                    int marks = sc.nextInt();
                    stuList.add(new Student(id, name, course, marks));
                    System.out.println("Student Added.");
                    break;
                case 2: // READ
                    if (stuList.isEmpty()) {
                        System.out.println("No students to display.");
                    } else {
                        for (Student s : stuList) {
                            s.display();
                        }
                    }
                    break;
                case 3: // UPDATE
                    System.out.print("Enter ID to update marks: ");
                    int uid = sc.nextInt();
                    boolean found = false;
                    for (Student s : stuList) {
                        if (s.id == uid) {
                            System.out.print("Current Marks = " + s.marks +
                                ". Enter New Marks: ");
                            s.marks = sc.nextInt();
                            System.out.println("Marks Updated.");
                            found = true;
                            break;
                        }
                    }
                }
            }
        }
    }
}

```

```

        if (!found) {
            System.out.println("Student not found.");
        }
        break;
    case 4: // DELETE
        System.out.print("Enter ID to delete: ");
        int did = sc.nextInt();
        Student toRemove = null;
        for (Student s : stuList) {
            if (s.id == did) {
                toRemove = s;
                break;
            }
        }
        if (toRemove != null) {
            stuList.remove(toRemove);
            System.out.println("Student removed.");
        } else {
            System.out.println("Student not found.");
        }
        break;
    case 5:
        System.out.println("Thank you!");
        break;
    default:
        System.out.println("Invalid choice.");
    }
} while (choice != 5);
sc.close();
}
}

===== 3) Restaurant Menu - List (ArrayList) =====
// File: RestaurantMenuCRUD.java
package com.collectiondemo;
import java.util.*;
// ----- MODEL CLASS -----
class MenuItem {
    int id;
    String name;
    double price;
    public MenuItem(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
    void display() {
        System.out.println("-----");
        System.out.println("Item ID    : " + id);
        System.out.println("Item Name  : " + name);
        System.out.println("Price     : " + price);
        System.out.println("-----");
    }
}
// ----- MAIN CLASS -----
public class RestaurantMenuCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<MenuItem> menu = new ArrayList<>();
        int choice;
        do {
            System.out.println("\n==== RESTAURANT MENU (ArrayList) ====");
            System.out.println("1. Add Menu Item (CREATE)");
            System.out.println("2. View All Items (READ)");
            System.out.println("3. Update Price by ID (UPDATE)");

```

```

System.out.println("4. Delete Item by ID (DELETE)");
System.out.println("5. Exit");
System.out.print("Enter Choice: ");
choice = sc.nextInt();
switch (choice) {
    case 1: // CREATE
        System.out.print("Enter Item ID   : ");
        int id = sc.nextInt();
        System.out.print("Enter Item Name : ");
        String name = sc.next();
        System.out.print("Enter Price   : ");
        double price = sc.nextDouble();
        menu.add(new MenuItem(id, name, price));
        System.out.println("Item Added.");
        break;
    case 2: // READ
        if (menu.isEmpty()) {
            System.out.println("No items in menu.");
        } else {
            for (MenuItem m : menu) {
                m.display();
            }
        }
        break;
    case 3: // UPDATE
        System.out.print("Enter Item ID to update price: ");
        int uid = sc.nextInt();
        boolean found = false;
        for (MenuItem m : menu) {
            if (m.id == uid) {
                System.out.print("Current Price = " + m.price +
                                ". Enter New Price: ");
                m.price = sc.nextDouble();
                System.out.println("Price Updated.");
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Item not found.");
        }
        break;
    case 4: // DELETE
        System.out.print("Enter Item ID to delete: ");
        int did = sc.nextInt();
        MenuItem toRemove = null;
        for (MenuItem m : menu) {
            if (m.id == did) {
                toRemove = m;
                break;
            }
        }
        if (toRemove != null) {
            menu.remove(toRemove);
            System.out.println("Item removed.");
        } else {
            System.out.println("Item not found.");
        }
        break;
    case 5:
        System.out.println("Thank you, visit again!");
        break;
    default:
        System.out.println("Invalid choice.");
}

```

```

        }
    } while (choice != 5);
    sc.close();
}
}

===== 4) Library Books - Set (HashSet) =====
// File: LibraryBookSetCRUD.java
package com.collectiondemo;
import java.util.*;
// ----- MODEL CLASS -----
class Book {
    int id;
    String title;
    String author;
    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }
    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Book)) return false;
        Book b = (Book) obj;
        return this.id == b.id;
    }
    void display() {
        System.out.println("-----");
        System.out.println("Book ID    : " + id);
        System.out.println("Title     : " + title);
        System.out.println("Author    : " + author);
        System.out.println("-----");
    }
}

// ----- MAIN CLASS -----
public class LibraryBookSetCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Set<Book> bookSet = new HashSet<>();
        int choice;
        do {
            System.out.println("\n==== LIBRARY BOOKS (HashSet) ====");
            System.out.println("1. Add Book (CREATE)");
            System.out.println("2. View All Books (READ)");
            System.out.println("3. Update Title by ID (UPDATE)");
            System.out.println("4. Delete Book by ID (DELETE)");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1: // CREATE
                    System.out.print("Enter Book ID    : ");
                    int id = sc.nextInt();
                    System.out.print("Enter Title      : ");
                    String title = sc.next();
                    System.out.print("Enter Author    : ");
                    String author = sc.next();
                    boolean exists = bookSet.contains(new Book(id, title, author));
                    if (exists) {
                        System.out.println("Book with this ID already exists.");
                    }

```

```

        } else {
            bookSet.add(new Book(id, title, author));
            System.out.println("Book Added.");
        }
        break;
    case 2: // READ
        if (bookSet.isEmpty()) {
            System.out.println("No books in library.");
        } else {
            for (Book b : bookSet) {
                b.display();
            }
        }
        break;
    case 3: // UPDATE
        System.out.print("Enter Book ID to update title: ");
        int uid = sc.nextInt();
        Book toUpdate = null;
        for (Book b : bookSet) {
            if (b.id == uid) {
                toUpdate = b;
                break;
            }
        }
        if (toUpdate != null) {
            System.out.print("Enter New Title: ");
            String newTitle = sc.next();
            toUpdate.title = newTitle;
            System.out.println("Title Updated.");
        } else {
            System.out.println("Book not found.");
        }
        break;
    case 4: // DELETE
        System.out.print("Enter Book ID to delete: ");
        int did = sc.nextInt();
        Book toRemove = null;
        for (Book b : bookSet) {
            if (b.id == did) {
                toRemove = b;
                break;
            }
        }
        if (toRemove != null) {
            bookSet.remove(toRemove);
            System.out.println("Book removed.");
        } else {
            System.out.println("Book not found.");
        }
        break;
    case 5:
        System.out.println("Thank you!");
        break;
    default:
        System.out.println("Invalid choice.");
    }
} while (choice != 5);
sc.close();
}

}

===== 5) Course Registration - Set (HashSet of Courses) =====
// File: CourseSetCRUD.java
package com.collectiondemo;
import java.util.*;

```

```

public class CourseSetCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Set<String> courses = new HashSet<>();
        int choice;
        do {
            System.out.println("\n==== COURSE REGISTRATION (HashSet) =====");
            System.out.println("1. Add Course (CREATE)");
            System.out.println("2. View All Courses (READ)");
            System.out.println("3. Remove Course (DELETE)");
            System.out.println("4. Check if Course Exists (READ)");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Course Name: ");
                    String c = sc.next();
                    if (courses.add(c)) {
                        System.out.println("Course Added.");
                    } else {
                        System.out.println("Course already exists.");
                    }
                    break;
                case 2:
                    if (courses.isEmpty()) {
                        System.out.println("No courses registered.");
                    } else {
                        System.out.println("Registered Courses:");
                        for (String s : courses) {
                            System.out.println("- " + s);
                        }
                    }
                    break;
                case 3:
                    System.out.print("Enter Course Name to remove: ");
                    String rc = sc.next();
                    if (courses.remove(rc)) {
                        System.out.println("Course removed.");
                    } else {
                        System.out.println("Course not found.");
                    }
                    break;
                case 4:
                    System.out.print("Enter Course Name to search: ");
                    String scourse = sc.next();
                    if (courses.contains(scourse)) {
                        System.out.println("Course is available.");
                    } else {
                        System.out.println("Course not found.");
                    }
                    break;
                case 5:
                    System.out.println("Thank you!");
                    break;
                default:
                    System.out.println("Invalid choice.");
            }
        } while (choice != 5);
        sc.close();
    }
}

===== 6) Product Inventory - Map (HashMap) =====
// File: ProductMapCRUD.java

```



```

package com.collectiondemo;
import java.util.*;
// ----- MODEL CLASS -----
class Product {
    int id;
    String name;
    int quantity;
    double price;
    public Product(int id, String name, int quantity, double price) {
        this.id = id;
        this.name = name;
        this.quantity = quantity;
        this.price = price;
    }
    void display() {
        System.out.println("-----");
        System.out.println("Product ID    : " + id);
        System.out.println("Name        : " + name);
        System.out.println("Quantity    : " + quantity);
        System.out.println("Price       : " + price);
        System.out.println("-----");
    }
}
// ----- MAIN CLASS -----
public class ProductMapCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Map<Integer, Product> productMap = new HashMap<>();
        int choice;
        do {
            System.out.println("\n==== PRODUCT INVENTORY (HashMap) ====");
            System.out.println("1. Add Product (CREATE)");
            System.out.println("2. View All Products (READ)");
            System.out.println("3. Update Quantity by ID (UPDATE)");
            System.out.println("4. Delete Product by ID (DELETE)");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Product ID    : ");
                    int id = sc.nextInt();
                    System.out.print("Enter Name        : ");
                    String name = sc.next();
                    System.out.print("Enter Quantity    : ");
                    int qty = sc.nextInt();
                    System.out.print("Enter Price       : ");
                    double price = sc.nextDouble();
                    if (productMap.containsKey(id)) {
                        System.out.println("Product ID already exists.");
                    } else {
                        productMap.put(id, new Product(id, name, qty, price));
                        System.out.println("Product Added.");
                    }
                    break;
                case 2:
                    if (productMap.isEmpty()) {
                        System.out.println("No products in inventory.");
                    } else {
                        for (Product p : productMap.values()) {
                            p.display();
                        }
                    }
                    break;
            }
        }
    }
}

```

```

        case 3:
            System.out.print("Enter Product ID to update quantity: ");
            int uid = sc.nextInt();
            Product p = productMap.get(uid);
            if (p != null) {
                System.out.print("Current Qty = " + p.quantity + ". Enter New Qty: ");
                p.quantity = sc.nextInt();
                System.out.println("Quantity Updated.");
            } else {
                System.out.println("Product not found.");
            }
            break;
        case 4:
            System.out.print("Enter Product ID to delete: ");
            int did = sc.nextInt();
            if (productMap.remove(did) != null) {
                System.out.println("Product removed.");
            } else {
                System.out.println("Product not found.");
            }
            break;
        case 5:
            System.out.println("Thank you!");
            break;
        default:
            System.out.println("Invalid choice.");
    }
} while (choice != 5);
sc.close();
}
}

===== 7) Bank Accounts - Map (TreeMap) =====
// File: BankAccountMapCRUD.java
package com.collectiondemo;
import java.util.*;
// ----- MODEL CLASS -----
class Account {
    long accNo;
    String name;
    double balance;
    public Account(long accNo, String name, double balance) {
        this.accNo = accNo;
        this.name = name;
        this.balance = balance;
    }
    void display() {
        System.out.println("-----");
        System.out.println("Acc No    : " + accNo);
        System.out.println("Name      : " + name);
        System.out.println("Balance   : " + balance);
        System.out.println("-----");
    }
}
// ----- MAIN CLASS -----
public class BankAccountMapCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Map<Long, Account> accMap = new TreeMap<>(); // sorted by accNo
        int choice;
        do {
            System.out.println("\n==== BANK ACCOUNTS (TreeMap) ====");
            System.out.println("1. Open Account (CREATE)");
            System.out.println("2. View All Accounts (READ)");
            System.out.println("3. Deposit (UPDATE)");

```

```

System.out.println("4. Close Account (DELETE)");
System.out.println("5. Exit");
System.out.print("Enter Choice: ");
choice = sc.nextInt();
switch (choice) {
    case 1:
        System.out.print("Enter Account No : ");
        long accNo = sc.nextLong();
        System.out.print("Enter Name      : ");
        String name = sc.next();
        System.out.print("Enter Opening Bal: ");
        double bal = sc.nextDouble();
        if (accMap.containsKey(accNo)) {
            System.out.println("Account already exists.");
        } else {
            accMap.put(accNo, new Account(accNo, name, bal));
            System.out.println("Account Opened.");
        }
        break;
    case 2:
        if (accMap.isEmpty()) {
            System.out.println("No accounts found.");
        } else {
            for (Account a : accMap.values()) {
                a.display();
            }
        }
        break;
    case 3:
        System.out.print("Enter Account No for deposit: ");
        long dacc = sc.nextLong();
        Account a = accMap.get(dacc);
        if (a != null) {
            System.out.print("Enter Amount to deposit: ");
            double amt = sc.nextDouble();
            a.balance += amt;
            System.out.println("Balance Updated. New Balance = " + a.balance);
        } else {
            System.out.println("Account not found.");
        }
        break;
    case 4:
        System.out.print("Enter Account No to close: ");
        long cacc = sc.nextLong();
        if (accMap.remove(cacc) != null) {
            System.out.println("Account closed.");
        } else {
            System.out.println("Account not found.");
        }
        break;
    case 5:
        System.out.println("Thank you for banking with us!");
        break;
    default:
        System.out.println("Invalid choice.");
}
} while (choice != 5);
sc.close();
}
}

```

```

===== 8) Hospital Patient Queue - Queue (LinkedList) =====
// File: HospitalQueueCRUD.java
package com.collectiondemo;
import java.util.*;

```

```
// ----- MODEL CLASS -----
class Patient {
    int id;
    String name;
    String problem;
    public Patient(int id, String name, String problem) {
        this.id = id;
        this.name = name;
        this.problem = problem;
    }
    void display() {
        System.out.println("[ " + id + " ] " + name + " - " + problem);
    }
}

// ----- MAIN CLASS -----
public class HospitalQueueCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Queue<Patient> queue = new LinkedList<>();
        int choice;
        do {
            System.out.println("\n==== HOSPITAL PATIENT QUEUE (LinkedList) ====");
            System.out.println("1. Add Patient (Enqueue / CREATE)");
            System.out.println("2. Call Next Patient (Dequeue / DELETE)");
            System.out.println("3. View Waiting Patients (READ)");
            System.out.println("4. Peek First Patient (READ)");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Patient ID   : ");
                    int id = sc.nextInt();
                    System.out.print("Enter Patient Name : ");
                    String name = sc.next();
                    System.out.print("Enter Problem      : ");
                    String prob = sc.next();
                    queue.offer(new Patient(id, name, prob));
                    System.out.println("Patient added to queue.");
                    break;
                case 2:
                    Patient next = queue.poll();
                    if (next != null) {
                        System.out.print("Calling Patient: ");
                        next.display();
                    } else {
                        System.out.println("No patients in queue.");
                    }
                    break;
                case 3:
                    if (queue.isEmpty()) {
                        System.out.println("No patients waiting.");
                    } else {
                        System.out.println("Waiting Patients:");
                        for (Patient p : queue) {
                            p.display();
                        }
                    }
                    break;
                case 4:
                    Patient first = queue.peek();
                    if (first != null) {
                        System.out.print("First Patient: ");
                        first.display();
                    }
            }
        } while (choice != 5);
    }
}
```

```

        } else {
            System.out.println("No patients in queue.");
        }
        break;
    case 5:
        System.out.println("Clinic closed for the day.");
        break;
    default:
        System.out.println("Invalid choice.");
    }
} while (choice != 5);
sc.close();
}
}

===== 9) Helpdesk Ticket Queue - PriorityQueue =====
// File: HelpdeskTicketQueueCRUD.java
package com.collectiondemo;
import java.util.*;
// ----- MODEL CLASS -----
class Ticket {
    int id;
    String issue;
    int priority; // 1 = High, 2 = Medium, 3 = Low
    public Ticket(int id, String issue, int priority) {
        this.id = id;
        this.issue = issue;
        this.priority = priority;
    }
    void display() {
        System.out.println("Ticket[" + id + "] (" + priority + ") - " + issue);
    }
}
// ----- MAIN CLASS -----
public class HelpdeskTicketQueueCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        PriorityQueue<Ticket> pq =
            new PriorityQueue<>(Comparator.comparingInt(t -> t.priority));
        int choice;
        do {
            System.out.println("\n==== HELPDESK TICKET QUEUE (PriorityQueue) =====");
            System.out.println("1. Raise Ticket (CREATE)");
            System.out.println("2. Resolve Next Ticket (DELETE)");
            System.out.println("3. View All Tickets (READ)");
            System.out.println("4. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Ticket ID   : ");
                    int id = sc.nextInt();
                    System.out.print("Enter Issue       : ");
                    String issue = sc.next();
                    System.out.print("Enter Priority(1-High,2-Med,3-Low): ");
                    int pr = sc.nextInt();
                    pq.offer(new Ticket(id, issue, pr));
                    System.out.println("Ticket raised.");
                    break;
                case 2:
                    Ticket t = pq.poll();
                    if (t != null) {
                        System.out.print("Resolving Ticket: ");
                        t.display();
                    } else {

```

```

        System.out.println("No tickets in queue.");
    }
    break;
case 3:
    if (pq.isEmpty()) {
        System.out.println("No tickets in queue.");
    } else {
        System.out.println("Pending Tickets (by internal order):");
        for (Ticket tk : pq) {
            tk.display();
        }
    }
    break;
case 4:
    System.out.println("Helpdesk closed.");
    break;
default:
    System.out.println("Invalid choice.");
}
} while (choice != 4);
sc.close();
}
}

===== 10) Bus Reservation Waiting List - Queue (LinkedList) =====
// File: BusWaitlistQueueCRUD.java
package com.collectiondemo;
import java.util.*;
// ----- MODEL CLASS -----
class Passenger {
    int id;
    String name;
    String fromCity;
    String toCity;
    public Passenger(int id, String name, String fromCity, String toCity) {
        this.id = id;
        this.name = name;
        this.fromCity = fromCity;
        this.toCity = toCity;
    }
    void display() {
        System.out.println("[ " + id + " ] " + name + " ( " + fromCity + " -> " + toCity + " )");
    }
}
// ----- MAIN CLASS -----
public class BusWaitlistQueueCRUD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Queue<Passenger> waitlist = new LinkedList<>();
        int choice;
        do {
            System.out.println("\n==== BUS WAITLIST (LinkedList) ====");
            System.out.println("1. Add Passenger to Waitlist (CREATE)");
            System.out.println("2. Confirm Next Passenger (DELETE)");
            System.out.println("3. View Waitlist (READ)");
            System.out.println("4. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Passenger ID : ");
                    int id = sc.nextInt();
                    System.out.print("Enter Name : ");
                    String name = sc.next();
                    System.out.print("Enter From City : ");

```

```

        String from = sc.next();
        System.out.print("Enter To City          : ");
        String to = sc.next();
        waitlist.offer(new Passenger(id, name, from, to));
        System.out.println("Passenger added to waitlist.");
        break;
    case 2:
        Passenger p = waitlist.poll();
        if (p != null) {
            System.out.print("Confirming Seat for: ");
            p.display();
        } else {
            System.out.println("No passengers in waitlist.");
        }
        break;
    case 3:
        if (waitlist.isEmpty()) {
            System.out.println("Waitlist is empty.");
        } else {
            System.out.println("Current Waitlist:");
            for (Passenger ps : waitlist) {
                ps.display();
            }
        }
        break;
    case 4:
        System.out.println("Bus booking system closed.");
        break;
    default:
        System.out.println("Invalid choice.");
    }
} while (choice != 4);
sc.close();
}
}

```